

目录

数据结构选型与技巧	1
线段树分治	1
小集合并到大集合	2
坐标和值域	2
可撤销与可持久化	2
左偏树	2
PBDS 有序集合	3
虚树	4
Link-Cut Tree	5
点分治：树上距离存在性	9
动态开点线段树合并	11
李超线段树	12
莫队算法	15
静态区间莫队	15
带修改莫队	16
树上莫队	17
笛卡尔树	17
单调栈线性构造	17
为什么能做 RMQ	18
可持久化线段树与动态主席树	18
静态版本：前缀版本主席树	18
动态主席树：树状数组套权值线段树	20
线段树分治模板	22
点分树：动态最近标记点	24
DSU on tree	27
长链剖分与 $O(1)$ 级祖先	28
重链剖分	31
带权并查集 / 势能并查集	32
树分块（王室联邦）	34

数据结构选型与技巧

线段树分治

若一个对象只在时间区间 $[l, r]$ 内有效，把它放到覆盖该区间的 $O(\log q)$ 个线段树节点。DFS 时间线段树时进入节点加入对象，离开节点撤销，即可把动态加删转成“只加入 + 回滚”。

常见搭配是可撤销并查集。回滚结构不能使用普通路径压缩；按大小合并保证树高 $O(\log n)$ ，并在栈中记录每次真正发生的修改。总复杂度通常为 $O((q+k)\log q \log n)$ ，其中 k 是对象有效区间数。

小集合并到大集合

合并两个集合时总把较小集合的元素逐个移入较大集合。一个元素所在集合的大小每次至少翻倍，因此每个元素至多被移动 $O(\log n)$ 次。该思想适用于 DSU on tree、启发式合并 **set/map**、可并堆外的频率统计等。

注意“按大小合并”只约束集合内容的搬运方向；若还需要保留树形父子关系或答案顺序，不能随意交换语义角色。

坐标和值域

- 只比较大小、排名或区间计数时，先离散化再使用树状数组/线段树。
- 需要在线插入任意坐标并查询最值时，考虑动态开点线段树或李超树。
- 需要第 k 小且版本可持久化时，考虑主席树；只有合并而无历史查询时，可合并线段树通常更直接。
- 所有下标、闭开区间和空区间约定应写在模板入口处，尤其不要混用 $[l,r]$ 与 $[l,r)$ 。

可撤销与可持久化

可撤销结构保存修改日志，适合 DFS 式进入/退出；可持久化结构保留每个版本的根，适合随机访问历史。两者都要求更新只触碰少量位置。带懒标记的数据结构在持久化时必须复制所有会被下传修改的节点，否则旧版本会被污染。

左偏树

可合并优先队列。每个 **push** 返回一个新堆根，**merge** 会破坏性地合并两个根。默认是小根堆。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

template<class Value, class Compare = less<Value>>
class LeftistHeap {
    struct Node {
        Value value{};
        int child[2]{0, 0};
        int null_path_length = 0;
    };

    vector<Node> nodes{Node{}};
    Compare compare;

public:
    int merge(int lhs, int rhs) {
        if (lhs == 0 || rhs == 0) return lhs | rhs;
        if (compare(nodes[rhs].value, nodes[lhs].value)) swap(lhs, rhs);
        nodes[lhs].child[1] = merge(nodes[lhs].child[1], rhs);
        int left = nodes[lhs].child[0];
        int right = nodes[lhs].child[1];
        if (nodes[left].null_path_length < nodes[right].null_path_length) {
            swap(nodes[lhs].child[0], nodes[lhs].child[1]);
        }
    }
};
```

```

        nodes[lhs].null_path_length =
            nodes[nodes[lhs].child[1]].null_path_length + 1;
        return lhs;
    }

    int push(int root, const Value& value) {
        int created = (int)nodes.size();
        nodes.push_back(Node{value, {0, 0}, 1});
        return merge(root, created);
    }

    const Value& top(int root) const {
        assert(root != 0);
        return nodes[root].value;
    }

    int pop(int root) {
        assert(root != 0);
        return merge(nodes[root].child[0], nodes[root].child[1]);
    }

    bool empty(int root) const {
        return root == 0;
    }
};

```

合并、插入和删除堆顶的最坏复杂度均为 $O(\log n)$ 。合并后旧根不应再作为独立堆使用。

PBDS 有序集合

GNU PBDS 的 `tree` 支持排名和第 k 小，接口不是标准 C++，只能在提供 `libstdc++` PBDS 的评测环境使用。

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

template<class Value>
using OrderedSet = tree<
    Value, null_type, less<Value>, rb_tree_tag,
    tree_order_statistics_node_update>;

template<class Value>
class OrderedMultiSet {
    using Key = pair<Value, int>;
    tree<Key, null_type, less<Key>, rb_tree_tag,
        tree_order_statistics_node_update> data;
    int next_id = 0;

public:
    void insert(const Value& value) {

```

```

        data.insert({value, next_id++});
    }

    bool erase_one(const Value& value) {
        auto iterator = data.lower_bound({value, numeric_limits<int>::min()});
        if (iterator == data.end() || iterator->first != value) return false;
        data.erase(iterator);
        return true;
    }

    int count_less(const Value& value) const {
        return (int)data.order_of_key(
            {value, numeric_limits<int>::min()});
    }

    int count_less_equal(const Value& value) const {
        return (int)data.order_of_key(
            {value, numeric_limits<int>::max()});
    }

    optional<Value> kth(int rank) const {
        if (rank < 0 || rank >= (int)data.size()) return nullopt;
        return data.find_by_order(rank)->first;
    }

    optional<Value> predecessor(const Value& value) const {
        int rank = count_less(value);
        return rank == 0 ? nullopt : kth(rank - 1);
    }

    optional<Value> successor(const Value& value) const {
        int rank = count_less_equal(value);
        return rank == (int)data.size() ? nullopt : kth(rank);
    }

    int size() const {
        return (int)data.size();
    }
};

```

对于 `OrderedSet<int> values`:

- `values.order_of_key(x)` 是严格小于 x 的元素个数;
- `values.find_by_order(k)` 返回 0-based 第 k 小的迭代器, 越界时等于 `end()`。

多重集合通过唯一编号区分相同值。编号只递增不复用; 若插入次数可能超过 `int`, 把编号改为 `long long`。

虚树

给定树上的关键点, 只保留关键点及连接它们所需的 LCA。下面函数返回虚树根、节点列表和有向父子边; 依赖预处理好的 DFS 序 `dfn`、`lca` 与祖先判断 `is_ancestor`。

```

#include <bits/stdc++.h>
using namespace std;

struct VirtualTree {
    int root = -1;
    vector<int> nodes;
    vector<pair<int, int>> edges;
};

template<class Lca, class IsAncestor>
VirtualTree build_virtual_tree(vector<int> key_nodes,
                              const vector<int>& dfn,
                              Lca lca, IsAncestor is_ancestor) {
    VirtualTree result;
    if (key_nodes.empty()) return result;
    sort(key_nodes.begin(), key_nodes.end(),
         [&](int lhs, int rhs) { return dfn[lhs] < dfn[rhs]; });
    key_nodes.erase(unique(key_nodes.begin(), key_nodes.end()), key_nodes.end());

    int original_size = (int)key_nodes.size();
    for (int i = 1; i < original_size; ++i) {
        key_nodes.push_back(lca(key_nodes[i - 1], key_nodes[i]));
    }
    sort(key_nodes.begin(), key_nodes.end(),
         [&](int lhs, int rhs) { return dfn[lhs] < dfn[rhs]; });
    key_nodes.erase(unique(key_nodes.begin(), key_nodes.end()), key_nodes.end());

    vector<int> stack;
    for (int node : key_nodes) {
        while (!stack.empty() && !is_ancestor(stack.back(), node)) {
            stack.pop_back();
        }
        if (!stack.empty()) result.edges.push_back({stack.back(), node});
        stack.push_back(node);
    }
    result.root = key_nodes[0];
    result.nodes = move(key_nodes);
    return result;
}

```

虚树点数不超过两倍关键点数减一，构造复杂度为 $O(k \log k)$ 。多次询问时只清空 `result.nodes` 中出现过的邻接表位置，不要每次清空整棵原树。

Link-Cut Tree

维护动态森林，支持连边、删边、单点修改和路径异或查询。点编号为 1-based；0 是空节点。所有公开函数均会维护辅助树状态。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

```

```

class LinkCutTree {
    struct Node {
        int child[2]{0, 0};
        int parent = 0;
        int value = 0;
        int path_xor = 0;
        bool reversed = false;
    };

    vector<Node> nodes;

    bool is_splay_root(int node) const {
        int parent = nodes[node].parent;
        return parent == 0 ||
            (nodes[parent].child[0] != node &&
             nodes[parent].child[1] != node);
    }

    void pull(int node) {
        nodes[node].path_xor =
            nodes[nodes[node].child[0]].path_xor ^
            nodes[node].value ^
            nodes[nodes[node].child[1]].path_xor;
    }

    void apply_reverse(int node) {
        if (node == 0) return;
        swap(nodes[node].child[0], nodes[node].child[1]);
        nodes[node].reversed = !nodes[node].reversed;
    }

    void push(int node) {
        if (!nodes[node].reversed) return;
        apply_reverse(nodes[node].child[0]);
        apply_reverse(nodes[node].child[1]);
        nodes[node].reversed = false;
    }

    void rotate(int node) {
        int parent = nodes[node].parent;
        int grandparent = nodes[parent].parent;
        int direction = nodes[parent].child[1] == node;
        int middle = nodes[node].child[direction ^ 1];

        if (!is_splay_root(parent)) {
            nodes[grandparent].child[nodes[grandparent].child[1] == parent] =
                node;
        }
        nodes[node].parent = grandparent;
        nodes[node].child[direction ^ 1] = parent;
        nodes[parent].parent = node;
        nodes[parent].child[direction] = middle;
        if (middle != 0) nodes[middle].parent = parent;
    }
}

```

```

        pull(parent);
        pull(node);
    }

    void splay(int node) {
        vector<int> ancestors{node};
        for (int current = node; !is_splay_root(current); ) {
            current = nodes[current].parent;
            ancestors.push_back(current);
        }
        while (!ancestors.empty()) {
            push(ancestors.back());
            ancestors.pop_back();
        }

        while (!is_splay_root(node)) {
            int parent = nodes[node].parent;
            int grandparent = nodes[parent].parent;
            if (!is_splay_root(parent)) {
                bool node_direction = nodes[parent].child[1] == node;
                bool parent_direction =
                    nodes[grandparent].child[1] == parent;
                rotate(node_direction == parent_direction ? parent : node);
            }
            rotate(node);
        }
    }

    void access(int node) {
        int preferred_child = 0;
        for (int current = node; current != 0;
            current = nodes[current].parent) {
            splay(current);
            nodes[current].child[1] = preferred_child;
            pull(current);
            preferred_child = current;
        }
        splay(node);
    }

public:
    explicit LinkCutTree(int node_count) : nodes(node_count + 1) {}

    void set_value(int node, int value) {
        access(node);
        nodes[node].value = value;
        pull(node);
    }

    void make_root(int node) {
        access(node);
        apply_reverse(node);
    }

```

```

int find_root(int node) {
    access(node);
    while (true) {
        push(node);
        if (nodes[node].child[0] == 0) break;
        node = nodes[node].child[0];
    }
    splay(node);
    return node;
}

bool connected(int lhs, int rhs) {
    if (lhs == rhs) return true;
    make_root(lhs);
    return find_root(rhs) == lhs;
}

bool link(int lhs, int rhs) {
    make_root(lhs);
    if (find_root(rhs) == lhs) return false;
    nodes[lhs].parent = rhs;
    return true;
}

bool cut(int lhs, int rhs) {
    make_root(lhs);
    access(rhs);
    if (nodes[rhs].child[0] != lhs ||
        nodes[lhs].child[1] != 0) {
        return false;
    }
    nodes[rhs].child[0] = 0;
    nodes[lhs].parent = 0;
    pull(rhs);
    return true;
}

optional<int> query_path_xor(int lhs, int rhs) {
    if (!connected(lhs, rhs)) return nullopt;
    make_root(lhs);
    access(rhs);
    return nodes[rhs].path_xor;
}
};

```

每个操作的均摊复杂度为 $O(\log n)$ 。这里只维护可交换的路径异或；改为路径和、最大值等信息时修改 pull 即可。若维护有方向的非交换信息，需要同时保存正序和逆序聚合值。

点分治：树上距离存在性

给定非负边权树和若干询问，判断是否存在两个不同点的距离等于询问值。点分治在每个重心处只统计路径经过该重心的点对；同一子树中的距离必须先查询、后加入，避免重复计数。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class CentroidDistanceQueries {
    using i64 = long long;
    struct Edge {
        int to;
        i64 weight;
    };

    int node_count;
    i64 maximum_query;
    vector<vector<Edge>> graph;
    vector<int> subtree_size;
    vector<bool> removed;
    vector<i64> queries;
    vector<bool> answer;

    int calculate_size(int node, int parent) {
        subtree_size[node] = 1;
        for (const Edge& edge : graph[node]) {
            if (edge.to != parent && !removed[edge.to]) {
                subtree_size[node] += calculate_size(edge.to, node);
            }
        }
        return subtree_size[node];
    }

    int find_centroid(int node, int parent, int component_size) {
        for (const Edge& edge : graph[node]) {
            if (edge.to != parent && !removed[edge.to] &&
                subtree_size[edge.to] > component_size / 2) {
                return find_centroid(edge.to, node, component_size);
            }
        }
        return node;
    }

    void collect_distances(int node, int parent, i64 distance,
                          vector<i64>& distances) const {
        if (distance > maximum_query) return;
        distances.push_back(distance);
        for (const Edge& edge : graph[node]) {
            if (edge.to != parent && !removed[edge.to]) {
                collect_distances(edge.to, node, distance + edge.weight,
                                distances);
            }
        }
    }
};
```

```

    }

    void process_centroid(int centroid) {
        unordered_set<i64> previous_distances;
        previous_distances.insert(0);
        for (const Edge& edge : graph[centroid]) {
            if (removed[edge.to]) continue;
            vector<i64> distances;
            collect_distances(edge.to, centroid, edge.weight, distances);
            for (i64 distance : distances) {
                for (int query_id = 0;
                     query_id < (int)queries.size(); ++query_id) {
                    if (!answer[query_id] && queries[query_id] >= distance &&
                        previous_distances.contains(
                            queries[query_id] - distance)) {
                        answer[query_id] = true;
                    }
                }
            }
            previous_distances.insert(distances.begin(), distances.end());
        }
    }

    void decompose(int start) {
        int component_size = calculate_size(start, -1);
        int centroid = find_centroid(start, -1, component_size);
        removed[centroid] = true;
        process_centroid(centroid);
        for (const Edge& edge : graph[centroid]) {
            if (!removed[edge.to]) decompose(edge.to);
        }
    }

public:
    explicit CentroidDistanceQueries(int initial_node_count)
        : node_count(initial_node_count), graph(initial_node_count),
          subtree_size(initial_node_count), removed(initial_node_count) {}

    void add_edge(int lhs, int rhs, i64 weight) {
        assert(weight >= 0);
        graph[lhs].push_back({rhs, weight});
        graph[rhs].push_back({lhs, weight});
    }

    vector<bool> solve(vector<i64> query_values) {
        queries = move(query_values);
        answer.assign(queries.size(), false);
        fill(removed.begin(), removed.end(), false);
        maximum_query =
            queries.empty() ? -1 : *max_element(queries.begin(), queries.end());
        if (node_count > 0 && maximum_query >= 0) decompose(0);
        return answer;
    }
}

```

```
};
```

设询问数为 q , 复杂度为 $O(nq \log n)$, 适合 q 较小的版本。所有边权非负是剪枝 $\text{distance} > \text{maximum_query}$ 正确的前提; 负边权树不能使用该剪枝。

动态开点线段树合并

维护固定整数值域上的权值和。根编号 0 表示空树; merge 破坏性合并两个根, 合并后不能继续把右根当作独立版本使用。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class MergeableSegmentTree {
    using i64 = long long;
    struct Node {
        int child[2]{0, 0};
        i64 sum = 0;
    };

    int left_bound;
    int right_bound;
    vector<Node> nodes{Node{}};

    int create() {
        nodes.push_back(Node{});
        return (int)nodes.size() - 1;
    }

    int update(int root, int left, int right, int position, i64 delta) {
        if (root == 0) root = create();
        nodes[root].sum += delta;
        if (left == right) return root;
        int middle = left + (right - left) / 2;
        if (position <= middle) {
            nodes[root].child[0] = update(
                nodes[root].child[0], left, middle, position, delta);
        } else {
            nodes[root].child[1] = update(
                nodes[root].child[1], middle + 1, right, position, delta);
        }
        return root;
    }

    int merge_roots(int lhs, int rhs, int left, int right) {
        if (lhs == 0 || rhs == 0) return lhs | rhs;
        nodes[lhs].sum += nodes[rhs].sum;
        if (left != right) {
            int middle = left + (right - left) / 2;
            nodes[lhs].child[0] = merge_roots(
                nodes[lhs].child[0], nodes[rhs].child[0], left, middle);
        }
    }
};
```

```

        nodes[lhs].child[1] = merge_roots(
            nodes[lhs].child[1], nodes[rhs].child[1], middle + 1, right);
    }
    return lhs;
}

public:
    MergeableSegmentTree(int minimum_coordinate, int maximum_coordinate)
        : left_bound(minimum_coordinate), right_bound(maximum_coordinate) {
        assert(minimum_coordinate <= maximum_coordinate);
    }

    void add(int& root, int position, i64 delta) {
        assert(left_bound <= position && position <= right_bound);
        root = update(root, left_bound, right_bound, position, delta);
    }

    int merge(int lhs, int rhs) {
        return merge_roots(lhs, rhs, left_bound, right_bound);
    }

    i64 total(int root) const {
        return nodes[root].sum;
    }

    optional<int> kth(int root, i64 rank) const {
        if (rank <= 0 || rank > nodes[root].sum) return nullopt;
        int left = left_bound;
        int right = right_bound;
        while (left != right) {
            int middle = left + (right - left) / 2;
            int left_child = nodes[root].child[0];
            if (rank <= nodes[left_child].sum) {
                root = left_child;
                right = middle;
            } else {
                rank -= nodes[left_child].sum;
                root = nodes[root].child[1];
                left = middle + 1;
            }
        }
        return left;
    }
};

```

单点修改和第 k 小为 $O(\log C)$ 。合并复杂度与两个树同时存在节点的位置数有关；一批树总合并通常可按被消去节点总数分析。使用 `kth` 时所有叶子权值必须非负。

李超线段树

在整数横坐标区间上维护若干线段，并查询某个横坐标处的最大值。若函数值相同，返回编号较小的线段。直线和端点使用 `long double`；如果题目中的 $kx + b$ 能安全装入 `__int128`，可改用整数获得精确比较。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class LiChaoSegmentTree {
    using ld = long double;
    static constexpr ld eps = 1e-12L;

    struct Line {
        ld slope;
        ld intercept;
        int id;

        ld value(ld x) const {
            return slope * x + intercept;
        }
    };

    int left_bound;
    int right_bound;
    vector<int> tree;
    vector<Line> lines;

    bool better(int lhs_id, int rhs_id, int x) const {
        if (lhs_id == -1) return false;
        if (rhs_id == -1) return true;
        ld difference = lines[lhs_id].value(x) - lines[rhs_id].value(x);
        if (fabsl(difference) > eps) return difference > 0;
        return lines[lhs_id].id < lines[rhs_id].id;
    }

    void insert_line(int node, int left, int right, int line_id) {
        if (tree[node] == -1) {
            tree[node] = line_id;
            return;
        }
        int middle = left + (right - left) / 2;
        if (better(line_id, tree[node], middle)) {
            swap(line_id, tree[node]);
        }
        if (left == right) return;
        if (better(line_id, tree[node], left)) {
            insert_line(node * 2, left, middle, line_id);
        } else if (better(line_id, tree[node], right)) {
            insert_line(node * 2 + 1, middle + 1, right, line_id);
        }
    }

    void insert_segment(int node, int left, int right,
                        int query_left, int query_right, int line_id) {
        if (query_left <= left && right <= query_right) {
            insert_line(node, left, right, line_id);
            return;
        }
    }

```

```

    }
    int middle = left + (right - left) / 2;
    if (query_left <= middle) {
        insert_segment(node * 2, left, middle,
            query_left, query_right, line_id);
    }
    if (query_right > middle) {
        insert_segment(node * 2 + 1, middle + 1, right,
            query_left, query_right, line_id);
    }
}

public:
    LiChaoSegmentTree(int minimum_x, int maximum_x)
        : left_bound(minimum_x), right_bound(maximum_x),
          tree(4 * (maximum_x - minimum_x + 1), -1) {
        assert(minimum_x <= maximum_x);
    }

    int add_segment(ld slope, ld intercept,
        int segment_left, int segment_right) {
        assert(left_bound <= segment_left && segment_left <= segment_right);
        assert(segment_right <= right_bound);
        int line_id = (int)lines.size();
        lines.push_back({slope, intercept, line_id});
        insert_segment(1, left_bound, right_bound,
            segment_left, segment_right, line_id);
        return line_id;
    }

    optional<pair<ld, int>> query(int x) const {
        assert(left_bound <= x && x <= right_bound);
        int node = 1;
        int left = left_bound;
        int right = right_bound;
        int best_id = -1;
        while (true) {
            if (better(tree[node], best_id, x)) best_id = tree[node];
            if (left == right) break;
            int middle = left + (right - left) / 2;
            if (x <= middle) {
                node *= 2;
                right = middle;
            } else {
                node = node * 2 + 1;
                left = middle + 1;
            }
        }
        if (best_id == -1) return nullopt;
        return pair<ld, int>{lines[best_id].value(x), lines[best_id].id};
    }
};

```

插入一条覆盖整个值域的直线是 $O(\log C)$, 插入线段是 $O(\log^2 C)$, 单点查询是 $O(\log C)$, 其中 C 是离

散整数横坐标范围长度。若横坐标很大但只查询预先给出的若干位置，应先离散化，并在线段端点处用原横坐标求值。

莫队算法

莫队只负责安排询问顺序。答案结构由调用者维护：移动左、右端点时执行 `add(index)` 或 `remove(index)`，然后用 `get_answer()` 读取当前区间答案。

静态区间莫队

区间均为 0-based 闭区间。奇偶块反向排序右端点可以减少指针抖动。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

struct MoQuery {
    int left;
    int right;
    int id;
};

template<class Answer, class Add, class Remove, class GetAnswer>
vector<Answer> mo_algorithm(int array_size, vector<MoQuery> queries,
                           Add add, Remove remove,
                           GetAnswer get_answer) {
    int block_size = max(1, (int)sqrt(max(1, array_size)));
    sort(queries.begin(), queries.end(), [&](const MoQuery& lhs,
                                              const MoQuery& rhs) {
        int lhs_block = lhs.left / block_size;
        int rhs_block = rhs.left / block_size;
        if (lhs_block != rhs_block) return lhs_block < rhs_block;
        return lhs_block & 1
            ? lhs.right > rhs.right : lhs.right < rhs.right;
    });

    vector<Answer> answers(queries.size());
    int current_left = 0;
    int current_right = -1;
    for (const MoQuery& query : queries) {
        assert(0 <= query.left && query.left <= query.right);
        assert(query.right < array_size);
        while (current_left > query.left) add(--current_left);
        while (current_right < query.right) add(++current_right);
        while (current_left < query.left) remove(current_left++);
        while (current_right > query.right) remove(current_right--);
        answers[query.id] = get_answer();
    }
    return answers;
}
```

若单次增删为 $O(1)$ ，常见复杂度为 $O((n + q)\sqrt{n})$ 。查询编号必须恰好覆盖 $0, 1, \dots, q - 1$ 。

带修改莫队

询问额外记录它之前发生的修改次数。修改对象应同时保存旧值和新值，使前进与回滚都为 $O(1)$ 。

```
struct TimedMoQuery {
    int left;
    int right;
    int time;
    int id;
};

template<class Answer, class Add, class Remove,
        class Apply, class Rollback, class GetAnswer>
vector<Answer> mo_with_updates(
    int array_size, int update_count, vector<TimedMoQuery> queries,
    Add add, Remove remove, Apply apply, Rollback rollback,
    GetAnswer get_answer) {
    int block_size = max(1, (int)pow(max(1, array_size), 2.0 / 3.0));
    sort(queries.begin(), queries.end(), [&](const TimedMoQuery& lhs,
                                             const TimedMoQuery& rhs) {
        int lhs_left_block = lhs.left / block_size;
        int rhs_left_block = rhs.left / block_size;
        if (lhs_left_block != rhs_left_block) {
            return lhs_left_block < rhs_left_block;
        }
        int lhs_right_block = lhs.right / block_size;
        int rhs_right_block = rhs.right / block_size;
        if (lhs_right_block != rhs_right_block) {
            return lhs_right_block < rhs_right_block;
        }
        return lhs.time < rhs.time;
    });

    vector<Answer> answers(queries.size());
    int current_left = 0;
    int current_right = -1;
    int current_time = 0;
    for (const TimedMoQuery& query : queries) {
        assert(0 <= query.time && query.time <= update_count);
        while (current_left > query.left) add(--current_left);
        while (current_right < query.right) add(++current_right);
        while (current_left < query.left) remove(current_left++);
        while (current_right > query.right) remove(current_right--);
        while (current_time < query.time) {
            apply(current_time++, current_left, current_right);
        }
        while (current_time > query.time) {
            rollback(--current_time, current_left, current_right);
        }
        answers[query.id] = get_answer();
    }
    return answers;
}
```

`apply(update_id, left, right)` 应在修改位置位于当前区间时先删除旧值、写入新值、再加入新值；

rollback 反向执行。常见复杂度为 $O(n^{2/3}(n+q))$ 次状态改变。

树上莫队

把 DFS 欧拉序列记录为“进入一次、离开一次”。移动指针时翻转对应点是否在集合中：

- 若 $\text{lca}(u, v) = u$ ，使用区间 $[in_u, in_v]$ ；
- 否则使用 $[out_u, in_v]$ ，并在读取答案前临时加入 LCA；
- 必须先保证 $in_u \leq in_v$ 。

路径区间公式取决于欧拉序列的具体定义，复制模板前应用链、祖先关系和跨子树三类样例手动检查。

笛卡尔树

给定数组 $a_0, a_1, \dots, a_{n-1}$ ，最小笛卡尔树同时满足：

1. 中序遍历的顶点顺序恰好是原数组下标顺序；
2. 每个父亲的键值不大于儿子的键值。

当键值互不相同，这棵树唯一。下面对相等键值不弹栈，因此区间最小值有多个位置时，较靠左者更接近根；这个稳定规则使树也唯一。

单调栈线性构造

```
#include <bits/stdc++.h>
using namespace std;

struct CartesianTree {
    int root = -1;
    vector<int> parent;
    vector<int> left_child;
    vector<int> right_child;
};

template<class Value>
CartesianTree build_min_cartesian_tree(const vector<Value>& values) {
    int size = (int)values.size();
    CartesianTree tree;
    tree.parent.assign(size, -1);
    tree.left_child.assign(size, -1);
    tree.right_child.assign(size, -1);
    vector<int> stack;

    for (int index = 0; index < size; ++index) {
        int last_popped = -1;
        while (!stack.empty() && values[stack.back()] > values[index]) {
            last_popped = stack.back();
            stack.pop_back();
        }
        if (!stack.empty()) {
            tree.right_child[stack.back()] = index;
            tree.parent[index] = stack.back();
        }
    }
}
```

```

    if (last_popped != -1) {
        tree.left_child[index] = last_popped;
        tree.parent[last_popped] = index;
    }
    stack.push_back(index);
}
if (!stack.empty()) tree.root = stack.front();
return tree;
}

```

每个下标只入栈、出栈一次，时间和空间复杂度都是 $O(n)$ 。最大笛卡尔树只需把比较号改成 $<$ 。

为什么能做 RMQ

区间 $[l, r]$ 的最小值位置等于笛卡尔树上 $\text{lca}(l, r)$ 。原因是中序遍历保证 $[l, r]$ 对应两点之间的一段连续序列，而小根堆性质保证它们 LCA 的键值最小。

因此：

- 数组 RMQ 可以转成树上 LCA；
- 任意区间的最小值对分治结构，等价于以笛卡尔树根分割左右区间；
- 单调栈构造过程也常用于求每个元素作为区间最小值时能控制的最远左右边界。

递增数组会得到一条向右的链，递减数组会得到一条向左的链。后续 DFS 可能达到 $O(n)$ 深度，应使用迭代遍历或确保栈空间足够。

可持久化线段树与动态主席树

静态版本：前缀版本主席树

值域离散化为 $[0, m)$ 。每次单点增量只复制根到叶子的一条链；`roots[version]` 保存该版本的根。两个前缀版本相减即可查询区间频率与第 k 小。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;

class PersistentSegmentTree {
    struct Node {
        int left = 0;
        int right = 0;
        i64 sum = 0;
    };

    int coordinate_count;
    vector<Node> nodes{{}};
    vector<int> roots{0};

    int add(int previous, int left, int right, int position, i64 delta) {
        int current = (int)nodes.size();
        nodes.push_back(nodes[previous]);
        nodes[current].sum += delta;
    }
};

```

```

        if (right - left == 1) return current;
        int middle = left + (right - left) / 2;
        if (position < middle) {
            nodes[current].left =
                add(nodes[previous].left, left, middle, position, delta);
        } else {
            nodes[current].right =
                add(nodes[previous].right, middle, right, position, delta);
        }
        return current;
    }

i64 range_sum(int node, int left, int right,
              int query_left, int query_right) const {
    if (query_right <= left || right <= query_left || node == 0) return 0;
    if (query_left <= left && right <= query_right) return nodes[node].sum;
    int middle = left + (right - left) / 2;
    return range_sum(nodes[node].left, left, middle,
                    query_left, query_right) +
        range_sum(nodes[node].right, middle, right,
                    query_left, query_right);
}

public:
    explicit PersistentSegmentTree(int initial_coordinate_count)
        : coordinate_count(initial_coordinate_count) {
        assert(initial_coordinate_count > 0);
    }

    int add_version(int previous_version, int position, i64 delta = 1) {
        assert(0 <= previous_version && previous_version < (int)roots.size());
        assert(0 <= position && position < coordinate_count);
        roots.push_back(add(roots[previous_version], 0, coordinate_count,
                            position, delta));
        return (int)roots.size() - 1;
    }

i64 query_sum(int version, int left, int right) const {
    assert(0 <= version && version < (int)roots.size());
    assert(0 <= left && left <= right && right <= coordinate_count);
    return range_sum(roots[version], 0, coordinate_count, left, right);
}

i64 kth_between(int left_version, int right_version, i64 k) const {
    assert(0 <= left_version && left_version < (int)roots.size());
    assert(0 <= right_version && right_version < (int)roots.size());
    int left_root = roots[left_version];
    int right_root = roots[right_version];
    i64 total = nodes[right_root].sum - nodes[left_root].sum;
    assert(1 <= k && k <= total);

    int segment_left = 0;
    int segment_right = coordinate_count;

```

```

while (segment_right - segment_left > 1) {
    i64 left_count =
        nodes[nodes[right_root].left].sum -
        nodes[nodes[left_root].left].sum;
    int middle = segment_left + (segment_right - segment_left) / 2;
    if (k <= left_count) {
        left_root = nodes[left_root].left;
        right_root = nodes[right_root].left;
        segment_right = middle;
    } else {
        k -= left_count;
        left_root = nodes[left_root].right;
        right_root = nodes[right_root].right;
        segment_left = middle;
    }
}
return segment_left;
}
};

```

一次更新和查询均为 $O(\log m)$ ，每个版本新增 $O(\log m)$ 个结点。用于区间第 k 小时，第 i 个前缀版本必须从第 $i-1$ 个版本继续插入，且频率差不能为负。

动态主席树：树状数组套权值线段树

下面支持原数组单点修改以及区间 $[l, r]$ 第 k 小。所有初值和未来修改值必须提前放入 `possible_values` 完成离散化；查询返回真实值。

```

class DynamicChairmanTree {
    struct Node {
        int left = 0;
        int right = 0;
        int count = 0;
    };

    int size;
    vector<int> coordinates;
    vector<int> current_value;
    vector<int> bit_roots;
    vector<Node> nodes{{}};

    int coordinate(int value) const {
        int index = (int)(lower_bound(coordinates.begin(), coordinates.end(),
                                      value) - coordinates.begin());
        assert(index < (int)coordinates.size() && coordinates[index] == value);
        return index;
    }

    int update_node(int root, int left, int right,
                   int position, int delta) {
        if (root == 0) {
            root = (int)nodes.size();
            nodes.push_back({});
        }
    }
}

```

```

nodes[root].count += delta;
assert(nodes[root].count >= 0);
if (right - left == 1) return root;
int middle = left + (right - left) / 2;
if (position < middle) {
    int child = update_node(nodes[root].left, left, middle,
                           position, delta);
    nodes[root].left = child;
} else {
    int child = update_node(nodes[root].right, middle, right,
                           position, delta);
    nodes[root].right = child;
}
return root;
}

void add_position(int position, int value, int delta) {
    int value_index = coordinate(value);
    for (int index = position + 1; index <= size; index += index & -index) {
        bit_roots[index] = update_node(
            bit_roots[index], 0, (int)coordinates.size(), value_index, delta);
    }
}

vector<int> prefix_roots(int length) const {
    vector<int> result;
    for (int index = length; index > 0; index -= index & -index) {
        result.push_back(bit_roots[index]);
    }
    return result;
}

public:
    DynamicChairmanTree(const vector<int>& initial,
                       vector<int> possible_values)
        : size((int)initial.size()), current_value(initial),
          bit_roots(size + 1) {
        possible_values.insert(possible_values.end(),
                               initial.begin(), initial.end());
        sort(possible_values.begin(), possible_values.end());
        possible_values.erase(unique(possible_values.begin(), possible_values.end()),
                               possible_values.end());
        assert(!possible_values.empty());
        coordinates = move(possible_values);
        for (int position = 0; position < size; ++position) {
            add_position(position, current_value[position], 1);
        }
    }

    void assign(int position, int new_value) {
        assert(0 <= position && position < size);
        add_position(position, current_value[position], -1);
        current_value[position] = new_value;
    }

```

```

        add_position(position, current_value[position], 1);
    }

    int kth(int left, int right, int k) const {
        assert(0 <= left && left < right && right <= size);
        assert(1 <= k && k <= right - left);
        vector<int> positive = prefix_roots(right);
        vector<int> negative = prefix_roots(left);
        int segment_left = 0;
        int segment_right = (int)coordinates.size();

        while (segment_right - segment_left > 1) {
            int left_count = 0;
            for (int root : positive) left_count += nodes[nodes[root].left].count;
            for (int root : negative) left_count -= nodes[nodes[root].left].count;
            int middle = segment_left + (segment_right - segment_left) / 2;
            bool go_left = k <= left_count;
            if (!go_left) k -= left_count;
            for (int& root : positive) {
                root = go_left ? nodes[root].left : nodes[root].right;
            }
            for (int& root : negative) {
                root = go_left ? nodes[root].left : nodes[root].right;
            }
            if (go_left) segment_right = middle;
            else segment_left = middle;
        }
        return coordinates[segment_left];
    }
};

```

单点修改和区间第 k 小均为 $O(\log n \log m)$ 。update_node 不保存 vector 元素的引用跨越 push_back, 避免扩容后引用失效。

线段树分治模板

把在时间区间 $[l, r)$ 有效的事件放入覆盖该区间的 $O(\log q)$ 个结点。DFS 进入结点时应用事件，离开时按相反顺序撤销。动态连通性最常用的状态就是下面这个可撤销并查集。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

struct RollbackDsu {
    vector<int> parent, size;
    vector<pair<int, int>> history; // 被接入的根、另一个根原来的大小
    int component_count;

    explicit RollbackDsu(int n) : parent(n), size(n, 1), component_count(n) {
        iota(parent.begin(), parent.end(), 0);
    }
};

```

```

int find(int x) const {
    while (x != parent[x]) x = parent[x];
    return x;
}

bool merge(int x, int y) {
    x = find(x);
    y = find(y);
    if (x == y) {
        history.push_back({-1, 0});
        return false;
    }
    if (size[x] < size[y]) swap(x, y);
    history.push_back({y, size[x]});
    parent[y] = x;
    size[x] += size[y];
    --component_count;
    return true;
}

void undo() {
    assert(!history.empty());
    auto [child, old_size] = history.back();
    history.pop_back();
    if (child == -1) return;
    int root = parent[child];
    size[root] = old_size;
    parent[child] = child;
    ++component_count;
}

bool same(int x, int y) const { return find(x) == find(y); }
};

template<class Event>
class SegmentTreeOverTime {
    int time_count;
    vector<vector<Event>> events;

    void add_interval(int node, int left, int right,
                     int query_left, int query_right, const Event& event) {
        if (query_right <= left || right <= query_left) return;
        if (query_left <= left && right <= query_right) {
            events[node].push_back(event);
            return;
        }
        int middle = left + (right - left) / 2;
        add_interval(node * 2, left, middle, query_left, query_right, event);
        add_interval(node * 2 + 1, middle, right,
                     query_left, query_right, event);
    }

public:

```

```

explicit SegmentTreeOverTime(int initial_time_count)
: time_count(initial_time_count),
  events(max(1, 4 * initial_time_count)) {
    assert(initial_time_count >= 0);
}

void add_interval(int left, int right, const Event& event) {
    assert(0 <= left && left <= right && right <= time_count);
    if (left < right) add_interval(1, 0, time_count, left, right, event);
}

template<class Apply, class Rollback, class Answer>
void traverse(Apply apply, Rollback rollback, Answer answer) const {
    if (time_count == 0) return;
    auto dfs = [&](auto&& self, int node, int left, int right) -> void {
        for (const Event& event : events[node]) apply(event);
        if (right - left == 1) {
            answer(left);
        } else {
            int middle = left + (right - left) / 2;
            self(self, node * 2, left, middle);
            self(self, node * 2 + 1, middle, right);
        }
        for (auto iterator = events[node].rbegin();
             iterator != events[node].rend(); ++iterator) {
            rollback(*iterator);
        }
    };
    dfs(dfs, 1, 0, time_count);
}
};

```

并查集不能路径压缩，否则一次撤销会涉及整条路径；只按大小合并即可保证 `find` 为 $O(\log n)$ 。无论 `merge` 是否真的合并，模板都会向历史栈压入一项，因此每次事件离开时恰好调用一次 `undo()`。

若共有 k 个有效区间，则事件存储量与应用次数均为 $O(k \log q)$ 。配合可撤销并查集，动态连通性总复杂度为 $O(k \log q \log n)$ 。典型用法是把边作为 `Event`，进入时 `dsu.merge(edge.u, edge.v)`，退出时 `dsu.undo()`，在叶子回答当前时刻询问。

点分树：动态最近标记点

对原树做点分治，把每个重心连向递归子块的重心，得到高度 $O(\log n)$ 的点分树。对每个原树结点保存它到所有点分树祖先的距离；每个重心维护其管辖标记点到它的距离集合。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class CentroidNearestMarked {
    int size;
    vector<vector<int>>> graph;
    vector<bool> blocked;
    vector<int> subtree_size;

```



```

vector<int> traversal_parent;
vector<int> decomposition_parent;
vector<vector<pair<int, int>>> routes;
vector<multiset<int>> distances;
vector<bool> active;

void decompose(int entry, int parent_centroid) {
    vector<int> component;
    traversal_parent[entry] = -1;
    vector<int> stack{entry};
    while (!stack.empty()) {
        int node = stack.back();
        stack.pop_back();
        component.push_back(node);
        for (int next : graph[node]) {
            if (blocked[next] || next == traversal_parent[node]) continue;
            traversal_parent[next] = node;
            stack.push_back(next);
        }
    }

    for (int index = (int)component.size() - 1; index >= 0; --index) {
        int node = component[index];
        subtree_size[node] = 1;
        for (int next : graph[node]) {
            if (blocked[next] || traversal_parent[next] != node) continue;
            subtree_size[node] += subtree_size[next];
        }
    }

    int total = (int)component.size();
    int centroid = entry;
    for (int node : component) {
        int largest_part = total - subtree_size[node];
        for (int next : graph[node]) {
            if (blocked[next] || traversal_parent[next] != node) continue;
            largest_part = max(largest_part, subtree_size[next]);
        }
        if (2 * largest_part <= total) {
            centroid = node;
            break;
        }
    }

    decomposition_parent[centroid] = parent_centroid;
    struct DistanceState { int node, parent, distance; };
    vector<DistanceState> distance_stack{{centroid, -1, 0}};
    while (!distance_stack.empty()) {
        auto [node, parent, distance] = distance_stack.back();
        distance_stack.pop_back();
        routes[node].push_back({centroid, distance});
        for (int next : graph[node]) {
            if (blocked[next] || next == parent) continue;

```

```

        distance_stack.push_back({next, node, distance + 1});
    }
}

blocked[centroid] = true;
for (int next : graph[centroid]) {
    if (!blocked[next]) decompose(next, centroid);
}
}

public:
explicit CentroidNearestMarked(const vector<vector<int>>& tree)
: size((int)tree.size()), graph(tree), blocked(size),
  subtree_size(size), traversal_parent(size, -1),
  decomposition_parent(size, -1), routes(size), distances(size),
  active(size) {
    if (size > 0) decompose(0, -1);
}

void toggle(int node) {
    assert(0 <= node && node < size);
    active[node] = !active[node];
    for (auto [centroid, distance] : routes[node]) {
        if (active[node]) {
            distances[centroid].insert(distance);
        } else {
            auto iterator = distances[centroid].find(distance);
            assert(iterator != distances[centroid].end());
            distances[centroid].erase(iterator);
        }
    }
}

int nearest_distance(int node) const {
    assert(0 <= node && node < size);
    int answer = numeric_limits<int>::max();
    for (auto [centroid, distance] : routes[node]) {
        if (!distances[centroid].empty()) {
            answer = min(answer, distance + *distances[centroid].begin());
        }
    }
    return answer == numeric_limits<int>::max() ? -1 : answer;
}

const vector<int>& parents() const { return decomposition_parent; }
};

```

输入必须是连通无根树。建树时间与路线总空间为 $O(n \log n)$ ；工作数组在各子块之间复用，不能在每次 `decompose` 中重新初始化长度为 n 的数组，否则最坏会退化到 $O(n^2)$ 。一次开关会在 $O(\log n)$ 个 `multiset` 中插入/删除，复杂度 $O(\log^2 n)$ ；查询访问 $O(\log n)$ 个重心，每次只取集合最小值，复杂度 $O(\log n)$ 。

若只有“加入标记、永不删除”，每个重心只维护一个最小距离即可把修改也降到 $O(\log n)$ 。

DSU on tree

先处理所有轻儿子并清空贡献，再处理重儿子并保留贡献，最后把轻子树重新加入。每个结点作为轻子树成员被重新加入的次数至多 $O(\log n)$ 。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

template<class AddVertex, class RemoveVertex, class Report>
void dsu_on_tree(const vector<vector<int>>& tree, int root,
                AddVertex add_vertex, RemoveVertex remove_vertex,
                Report report) {
    int n = (int)tree.size();
    assert(0 <= root && root < n);
    vector<int> parent(n, -1), subtree_size(n, 1), heavy_child(n, -1);
    vector<int> order{root};
    for (int index = 0; index < (int)order.size(); ++index) {
        int node = order[index];
        for (int next : tree[node]) {
            if (next == parent[node]) continue;
            parent[next] = node;
            order.push_back(next);
        }
    }
    assert((int)order.size() == n);
    for (int index = n - 1; index >= 0; --index) {
        int node = order[index];
        for (int next : tree[node]) {
            if (parent[next] != node) continue;
            subtree_size[node] += subtree_size[next];
            if (heavy_child[node] == -1 ||
                subtree_size[next] > subtree_size[heavy_child[node]]) {
                heavy_child[node] = next;
            }
        }
    }

    vector<int> tin(n), tout(n), euler;
    euler.reserve(n);
    vector<pair<int, int>> stack{{root, 0}};
    while (!stack.empty()) {
        int node = stack.back().first;
        int& next_index = stack.back().second;
        if (next_index == 0) {
            tin[node] = (int)euler.size();
            euler.push_back(node);
        }
        if (next_index == (int)tree[node].size()) {
            tout[node] = (int)euler.size();
            stack.pop_back();
            continue;
        }
        int next = tree[node][next_index++];
```

```

    if (parent[next] == node) stack.push_back({next, 0});
}

auto add_subtree = [&](int node) {
    for (int index = tin[node]; index < tout[node]; ++index) {
        add_vertex(euler[index]);
    }
};

auto remove_subtree = [&](int node) {
    for (int index = tin[node]; index < tout[node]; ++index) {
        remove_vertex(euler[index]);
    }
};

auto solve = [&](auto&& self, int node, bool keep) -> void {
    for (int next : tree[node]) {
        if (parent[next] == node && next != heavy_child[node]) {
            self(self, next, false);
        }
    }
    if (heavy_child[node] != -1) self(self, heavy_child[node], true);
    for (int next : tree[node]) {
        if (parent[next] == node && next != heavy_child[node]) {
            add_subtree(next);
        }
    }
    add_vertex(node);
    report(node);
    if (!keep) remove_subtree(node);
};

solve(solve, root, false);
}

```

输入必须是连通无根树。若 `add_vertex/remove_vertex` 为 $O(1)$ ，总复杂度为 $O(n \log n)$ ，`report` 对每个结点调用一次。典型应用是求每棵子树的颜色频率、众数、出现次数达到阈值的颜色数。它适用于静态、以子树为单位的离线统计；带修改或任意路径查询通常应换用莫队、树链剖分或点分树。核心 `solve` 使用递归，链状树且 n 很大时应改成显式事件栈或确保栈空间足够。

长链剖分与 $O(1)$ 级祖先

重链剖分选择“子树最大”的儿子，长链剖分则选择“向下最深”的儿子。它主要用于与深度有关的问题：

- 预处理后 $O(1)$ 查询结点的第 k 个祖先；
- 合并形如 `dp[u][distance]` 的树 DP。重儿子的数组可直接把指针平移一格复用，轻儿子的数组总合并量可做到 $O(n)$ ；典型问题是统计每个点子树内各种深度的结点数、求最常见深度。

下面给出较容易直接复用的 $O(1)$ 级祖先版本。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class LongChainAncestors {

```

```

int size;
int logarithm;
vector<int> parent;
vector<int> depth;
vector<int> height;
vector<int> heavy_child;
vector<int> chain_top;
vector<vector<int>> binary_ancestor;
vector<vector<int>> downward;
vector<vector<int>> upward;

public:
LongChainAncestors(const vector<vector<int>>& tree, int root)
: size((int)tree.size()), logarithm(1), parent(size, -1), depth(size),
  height(size, 1), heavy_child(size, -1), chain_top(size),
  downward(size), upward(size) {
assert(0 <= root && root < size);
vector<int> order{root};
for (int index = 0; index < (int)order.size(); ++index) {
    int node = order[index];
    for (int next : tree[node]) {
        if (next == parent[node]) continue;
        parent[next] = node;
        depth[next] = depth[node] + 1;
        order.push_back(next);
    }
}
assert((int)order.size() == size);
for (int index = size - 1; index >= 0; --index) {
    int node = order[index];
    for (int next : tree[node]) {
        if (parent[next] != node) continue;
        if (heavy_child[node] == -1 ||
            height[next] > height[heavy_child[node]]) {
            heavy_child[node] = next;
        }
    }
    if (heavy_child[node] != -1) {
        height[node] = height[heavy_child[node]] + 1;
    }
}

chain_top[root] = root;
for (int node : order) {
    for (int next : tree[node]) {
        if (parent[next] != node) continue;
        chain_top[next] =
            next == heavy_child[node] ? chain_top[node] : next;
    }
}

while ((1 << logarithm) <= max(1, size)) ++logarithm;
binary_ancestor.assign(logarithm, vector<int>(size, -1));

```

```

binary_ancestor[0] = parent;
for (int level = 1; level < logarithm; ++level) {
    for (int node = 0; node < size; ++node) {
        int middle = binary_ancestor[level - 1][node];
        if (middle != -1) {
            binary_ancestor[level][node] =
                binary_ancestor[level - 1][middle];
        }
    }
}

for (int head : order) {
    if (chain_top[head] != head) continue;
    int node = head;
    while (node != -1) {
        downward[head].push_back(node);
        node = heavy_child[node];
    }
    upward[head].resize(downward[head].size(), -1);
    node = head;
    for (int distance = 0; distance < (int)upward[head].size();
        ++distance) {
        upward[head][distance] = node;
        if (node != -1) node = parent[node];
    }
}

int kth_ancestor(int node, int distance) const {
    assert(0 <= node && node < size && distance >= 0);
    if (distance > depth[node]) return -1;
    if (distance == 0) return node;

    int level = 31 - __builtin_clz((unsigned)distance);
    int jumped = binary_ancestor[level][node];
    int remaining = distance - (1 << level);
    int head = chain_top[jumped];
    int inside_chain = depth[jumped] - depth[head];
    if (remaining <= inside_chain) {
        return downward[head][inside_chain - remaining];
    }
    int above_head = remaining - inside_chain;
    assert(above_head < (int)upward[head].size());
    return upward[head][above_head];
}
};

```

上述实现预处理 $O(n \log n)$ 、空间 $O(n \log n)$ ，查询只做一次二进制跳跃和一次链内数组访问，为 $O(1)$ 。
`upward` 为每条长链只保存与链长相同的一段祖先；所有链长之和为 n 。

如果只需要普通 LCA 或路径加和，长链剖分没有优势，使用倍增或按子树大小的树链剖分更自然。它真正的价值是“深度/距离维 DP 的数组复用”和大量级祖先查询。

重链剖分

按子树大小选择重儿子，使任意根路径至多经过 $O(\log n)$ 条重链。path_vertices 把点路径拆成若干个 $[\text{left}, \text{right})$ 的 DFS 序区间，适合配合线段树做交换律成立的路径加、路径和或路径最值。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

struct HeavyLightDecomposition {
    int n, timer = 0;
    vector<int> parent, depth, size, heavy, head, position;

    HeavyLightDecomposition(const vector<vector<int>>& tree, int root = 0)
        : n((int)tree.size()), parent(n, -1), depth(n), size(n),
          heavy(n, -1), head(n), position(n) {
        assert(0 <= root && root < n);
        auto get_size = [&](auto&& self, int u, int p) -> void {
            parent[u] = p;
            size[u] = 1;
            for (int v : tree[u]) if (v != p) {
                depth[v] = depth[u] + 1;
                self(self, v, u);
                size[u] += size[v];
                if (heavy[u] == -1 || size[v] > size[heavy[u]]) heavy[u] = v;
            }
        };
        auto decompose = [&](auto&& self, int u, int top) -> void {
            head[u] = top;
            position[u] = timer++;
            if (heavy[u] != -1) self(self, heavy[u], top);
            for (int v : tree[u]) {
                if (v != parent[u] && v != heavy[u]) self(self, v, v);
            }
        };
        get_size(get_size, root, -1);
        decompose(decompose, root, root);
        assert(timer == n);
    }

    int lca(int u, int v) const {
        while (head[u] != head[v]) {
            if (depth[head[u]] < depth[head[v]]) swap(u, v);
            u = parent[head[u]];
        }
        return depth[u] < depth[v] ? u : v;
    }

    template<class Work>
    void path_vertices(int u, int v, Work work) const {
        while (head[u] != head[v]) {
            if (depth[head[u]] < depth[head[v]]) swap(u, v);
            work(position[head[u]], position[u] + 1);
            u = parent[head[u]];
        }
    }
};
```

```

    }
    if (depth[u] > depth[v]) swap(u, v);
    work(position[u], position[v] + 1);
}

template<class Work>
void path_edges(int u, int v, Work work) const {
    while (head[u] != head[v]) {
        if (depth[head[u]] < depth[head[v]]) swap(u, v);
        work(position[head[u]], position[u] + 1);
        u = parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    if (u != v) work(position[u] + 1, position[v] + 1);
}

pair<int, int> subtree(int u) const {
    return {position[u], position[u] + size[u]};
}
};

```

两次 DFS 为 $O(n)$ ；一条路径拆成 $O(\log n)$ 段。若线段树单段操作为 $O(\log n)$ ，路径操作就是 $O(\log^2 n)$ 。代码使用递归 DFS，链状树且 n 很大时需要留意栈空间。

上面的区间回调不保证按 $u \rightarrow v$ 的方向出现，所以只适合加法、最大值等满足交换律的操作。维护矩阵乘积或字符串这类有方向的信息时，需要分别保存 u 侧和 v 侧区间并反转其中一侧。边权通常存到较深端点；最后一段排除 LCA 的 `position[lca]` 即可。

直接接线段树时通常只写下面几行，其中线段树也统一使用 `[left, right)`：

```

// 点权路径和
long long answer = 0;
hld.path_vertices(u, v, [&](int left, int right) {
    answer += segment_tree.query(left, right);
});

// 子树加
auto [left, right] = hld.subtree(u);
segment_tree.add(left, right, delta);

// 边权路径查询：每条边的权值存在较深端点
long long answer = 0;
hld.path_edges(u, v, [&](int left, int right) {
    answer += segment_tree.query(left, right);
});

```

`path_edges` 会在最后一条链上直接排除 LCA，因此求和、最大值等操作都能使用；当 $u=v$ 时路径没有边，不会调用回调。

带权并查集 / 势能并查集

势能并查集维护形如

$$potential[y] - potential[x] = difference$$

的约束。weight[x] 表示 potential[x] - potential[parent[x]]；路径压缩时把沿途差值相加。它适合维护相对坐标、区间和关系、判定差分约束是否矛盾。这里只封装合并、连通性和差值查询。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

struct PotentialDsu {
    using i64 = long long;

    vector<int> parent, size;
    vector<i64> weight;

    explicit PotentialDsu(int n) : parent(n), size(n, 1), weight(n) {
        iota(parent.begin(), parent.end(), 0);
    }

    pair<int, i64> find(int x) {
        if (parent[x] == x) return {x, 0};
        int old_parent = parent[x];
        auto [root, parent_weight] = find(old_parent);
        __int128_t total = (__int128_t)weight[x] + parent_weight;
        assert(total >= numeric_limits<i64>::min());
        assert(total <= numeric_limits<i64>::max());
        parent[x] = root;
        weight[x] = (i64)total;
        return {root, weight[x]};
    }

    // 加入 potential[y] - potential[x] = difference。
    // 已连通时返回该约束是否与旧约束相容。
    bool add_constraint(int x, int y, i64 difference) {
        auto [root_x, weight_x] = find(x);
        auto [root_y, weight_y] = find(y);
        if (root_x == root_y) {
            return (__int128_t)weight_y - weight_x == difference;
        }

        if (size[root_x] >= size[root_y]) {
            // potential[root_y] - potential[root_x]
            __int128_t value = (__int128_t)difference + weight_x - weight_y;
            assert(value >= numeric_limits<i64>::min());
            assert(value <= numeric_limits<i64>::max());
            parent[root_y] = root_x;
            weight[root_y] = (i64)value;
            size[root_x] += size[root_y];
        } else {
            // potential[root_x] - potential[root_y]
            __int128_t value = (__int128_t)weight_y - weight_x - difference;
            assert(value >= numeric_limits<i64>::min());
            assert(value <= numeric_limits<i64>::max());
            parent[root_x] = root_y;
        }
    }
};
```

```

        weight[root_x] = (i64)value;
        size[root_y] += size[root_x];
    }
    return true;
}

// 未连通返回 nullopt, 否则返回 potential[y] - potential[x]。
optional<i64> difference(int x, int y) {
    auto [root_x, weight_x] = find(x);
    auto [root_y, weight_y] = find(y);
    if (root_x != root_y) return nullopt;
    __int128_t value = (__int128_t)weight_y - weight_x;
    assert(value >= numeric_limits<i64>::min());
    assert(value <= numeric_limits<i64>::max());
    return (i64)value;
}
};

```

合并公式最容易写反。若把 `root_y` 接到 `root_x`, 由

$$(weight_y + potential[root_y]) - (weight_x + potential[root_x]) = difference$$

立刻得到新边权 `difference + weight_x - weight_y`。建议现场忘记时按这行重推，不要背符号。

若关系是在模 k 意义下，只需令所有 `weight` 和 `difference` 都先规范到 $[0, k)$ ，把加减法结果再次取规范模；异或关系则把上述加减全部换成异或。普通整数版本不能直接拿来表达“只在模 k 下相等”的关系。

复杂度与普通并查集相同，均摊 $O(\alpha(n))$ 。这里的断言只防止势能累计溢出；题目若允许差值超出 `long long`，需要把 `weight` 本身改成 `__int128`。

树分块（王室联邦）

给定一棵树和块大小下界 B ，可以在 $O(n)$ 时间内把点划分成若干块。除整棵树不足 B 个点的情形外，每块大小均在 $[B, 3B)$ 内，块数不超过 $\lceil n/B \rceil$ 。每块还有一个核心点；核心不一定属于该块，但块内任意点走向核心时，核心之前的点仍在该块内。

DFS 时用栈保存尚未分块的点。进入点 `node` 时记录栈底；每处理完一个儿子，若这个儿子新留下的点连同此前未分块的兄弟子树已经达到 B 个，就全部弹出组成一块，并令 `node` 为核心。处理新儿子前栈中这部分少于 B 个，而一个儿子返回时至多新留下 B 个，所以新块小于 $2B$ ；DFS 结束后至多 B 个残余点并入最后一块，因此最后一块小于 $3B$ 。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

struct TreeBlocks {
    vector<int> block_id; // 每个点所属的块，编号从 0 开始
    vector<int> core;     // 每块的核心；核心不一定属于该块
};

TreeBlocks divide_tree_into_blocks(
    const vector<vector<int>>& tree, int block_size, int root = 0) {
    int n = (int)tree.size();

```

```

assert(block_size > 0);
if (n == 0) return {};
assert(0 <= root && root < n);

TreeBlocks result{vector<int>(n, -1), {}};
vector<int> pending;

auto dfs = [&](auto&& self, int node, int parent) -> void {
    int bottom = (int)pending.size();
    for (int next : tree[node]) {
        if (next == parent) continue;
        self(self, next, node);
        if ((int)pending.size() - bottom >= block_size) {
            int id = (int)result.core.size();
            result.core.push_back(node);
            while ((int)pending.size() > bottom) {
                result.block_id[pending.back()] = id;
                pending.pop_back();
            }
        }
        pending.push_back(node);
    }
};

dfs(dfs, root, -1);
if (result.core.empty()) result.core.push_back(root);
int last_id = (int)result.core.size() - 1;
while (!pending.empty()) {
    result.block_id[pending.back()] = last_id;
    pending.pop_back();
}
return result;
}

```

取 $B \approx \sqrt{n}$ 后，块数和每块大小都是 $O(\sqrt{n})$ 。典型用法是把整块信息预处理，只对路径两端所在块或零散点暴力；这是一种按点划分的静态技巧，不等同于重链剖分，也不自动支持在线修改。

注意：输入必须是连通树。代码使用递归 DFS，链很长且栈空间较小时需改成手写栈或增大程序栈。